



UFRJ



INSTITUTO DE
COMPUTAÇÃO
UFRJ

Programação de Computadores II

Laboratório 11

Métodos de Ordenação

Prof. Marcos Tomazzoli Leipnitz

- Considere a seguinte função de intercalação para resolver o exercício e o desafio.

```
// A função recebe vetores crescentes v[p..q-1]
// e v[q..r-1] e reorganiza v[p..r-1] em ordem
// crescente.

static void intercala (int p, int q, int r, int v[])
{
    int *w;
    w = malloc ((r-p) * sizeof (int));
    int i = p, j = q;
    int k = 0;

    while (i < q && j < r) {
        if (v[i] <= v[j]) w[k++] = v[i++];
        else w[k++] = v[j++];
    }
    while (i < q) w[k++] = v[i++];
    while (j < r) w[k++] = v[j++];
    for (i = p; i < r; ++i) v[i] = w[i-p];
    free (w);
}
```

Exercício

- As seguintes funções prometem rearranjar $v[p..r-1]$ em ordem crescente. Indique, para cada uma delas, se está correta ou não. Em caso negativo, aponte o(s) erro(s) para cada uma delas.

```
void mergesort1 (int p, int r, int v[]) {  
    if (p < r-1) {  
        int q = (p + r) / 2;  
        mergesort1 (p, q, v);  
        mergesort1 (q, r, v);  
        intercala (p, q+1, r, v); } }
```

```
void mergesort2 (int p, int r, int v[]) {  
    if (p < r) {  
        int q = (p + r) / 2;  
        mergesort2 (p, q, v);  
        mergesort2 (q, r, v);  
        intercala (p, q, r, v); } }
```

```
void mergesort3 (int p, int r, int v[]) {  
    if (p < r-1) {  
        int q = (p + r - 1) / 2;  
        mergesort3 (p, q, v);  
        mergesort3 (q, r, v);  
        intercala (p, q, r, v); } }
```

Exercício

- As seguintes funções prometem rearranjar $v[p..r-1]$ em ordem crescente. Indique, para cada uma delas, se está correta ou não. Em caso negativo, aponte o(s) erro(s) para cada uma delas.

```
void mergesort4 (int p, int r, int v[]) {  
    if (p < r-1) {  
        int q = (p + r) / 2;  
        mergesort4 (p, q-1, v);  
        mergesort4 (q-1, r, v);  
        intercala (p, q-1, r, v); } }
```

Em `mergesort4`, e se trocarmos “ $(p+r)/2$ ” por “ $(p+r+1)/2$ ”?

```
void mergesort5 (int p, int r, int v[]) {  
    if (p < r-1) {  
        q = r - 2;  
        mergesort5 (p, q, v);  
        if (v[r-2] > v[r-1]) {  
            int t = v[r-2];  
            v[r-2] = v[r-1];  
            v[r-1] = t; }  
        intercala (p, q, r, v); } }
```

```
void mergesort6 (int p, int r, int v[]) {  
    if (p < r-1) {  
        q = r - 1;  
        mergesort6 (p, q, v);  
        intercala (p, q, r, v); } }
```

Desafio 10

- Escreva uma versão iterativa do algoritmo *Merge Sort* para rearranjar um vetor $v[0 \dots n-1]$ em ordem crescente, com a assinatura:

void mergesort_iterativo(int n, int v[])

Dica: a cada iteração, o algoritmo deve **intercalar** dois blocos consecutivos de **b** elementos cada: o primeiro bloco com o segundo, o terceiro com o quarto, etc. A variável **b** assume os valores 1, 2, 4, 8, ...

Crie um programa (main) para testar a sua função. O tamanho e o conteúdo do vetor de teste devem ser lidos da entrada padrão e o resultado apresentado na tela.